LAB 22 — FILE HANDLING

Complete Reference & Concept Guide for C Programming

Practical 22A, 22B and 22C • File functions • Modes • Syntax • Examples • Common mistakes

Purpose of this guide

This guide focuses on the concepts and references needed to understand Lab 22 and write your own solutions. It explains what each function does, its syntax, when to use it, examples, file modes, algorithms, and how the concepts fit each practical question.

Lab 22 topic map

• **22A:** create/open/close a file; count characters, spaces, tabs and newlines; demonstrate fprintf(), fscanf(), fputc(), fgetc(), fseek() and rewind(); append files; copy a file.

• **22B:** print file contents in reverse order; capitalize the first letter of each word; merge alternate lines from two files.

• **22C:** capitalize every word's first letter; remove blank lines and insert a blank line after the third line.

How to use this guide

• Read the file pointer and fopen() sections first.

• Learn modes such as r, w, a and w+.

• Practise character functions before line functions.

• Use the concept map near the end to choose functions for each question.

• Keep input files in the same working directory as the running program.

1. Basic File Handling Concepts

What is file handling? File handling lets a C program store, read, append, and process data in a file. Unlike ordinary variables, a saved file can keep its data after the program finishes.

What is a file pointer? A file pointer connects your program to an opened file. Declare it with FILE *fp; after including stdio.h.

```
#include <stdio.h>

int main(void)
{
    FILE *fp;
    fp = fopen("data.txt", "r");

    if (fp == NULL)
    {
        printf("Cannot open file.\n");
        return 1;
    }

    fclose(fp);
    return 0;
}
```

Line-by-line

• #include provides standard file functions.

• FILE *fp; declares a pointer for file operations.

• fopen() opens the named file using a selected mode.

• NULL means the file could not be opened.

• fclose(fp) closes the file and releases the file resource.

2. fopen() and File Modes

Syntax: **FILE *fp = fopen("filename", "mode");**

Mode	Purpose	Important behaviour
"r"	Read	File must already exist.
"w"	Write	Creates file or erases existing contents.
"a"	Append	Writes at end; creates file if absent.
"r+"	Read and write	Existing file required; does not truncate.
"w+"	Read and write	Creates or truncates the file.
"a+"	Read and append	Creates if absent; writes at end.
"rb", "wb", "ab"	Binary modes	Useful for byte-oriented files; b matters on some systems.

Example: create a file and write a line

```
FILE *fp = fopen("note.txt", "w");

if (fp == NULL)
{
    printf("Cannot open file.\n");
    return 1;
}

fprintf(fp, "Hello File Handling!\n");
fclose(fp);
```

This creates note.txt if it does not exist. If it already exists, its old contents are replaced.

3. Essential File Functions — Syntax and Examples

fclose()

Syntax: **fclose(fp);** Closes an opened file. Always close files when finished.

fprintf() — formatted writing

Syntax: **fprintf(fp, "format", values);** Works like printf(), but sends output to a file.

```
FILE *fp = fopen("student.txt", "w");
if (fp != NULL)
{
    fprintf(fp, "Name: %s Age: %d\n", "Parth", 18);
    fclose(fp);
}
```

fscanf() — formatted reading

Syntax: **fscanf(fp, "format", &variables;);** Works like scanf(), but reads from a file. Use & for ordinary variables; a character array name is passed without &.

```
FILE *fp = fopen("student.txt", "r");
char name[30];
int age;

if (fp != NULL)
{
    if (fscanf(fp, "Name: %29s Age: %d", name, &age) == 2)
        printf("%s %d\n", name, age);
    fclose(fp);
}
```

fputc() — writing one character

Syntax: **fputc(character, fp);**

```
FILE *fp = fopen("letter.txt", "w");
if (fp != NULL)
{
    fputc('A', fp);
    fclose(fp);
}
```

fgetc() — reading one character

Syntax: **int ch = fgetc(fp);** Returns the next character as an int, or EOF when the end/error is reached. Use int, not char, for the reading variable.

```
FILE *fp = fopen("letter.txt", "r");
int ch;

if (fp != NULL)
{
    while ((ch = fgetc(fp)) != EOF)
        putchar(ch);
    fclose(fp);
}
```

4. File Position Functions

fseek() moves the file position. Syntax: **fseek(fp, offset, origin);**

Origin	Meaning
SEEK_SET	Beginning of the file
SEEK_CUR	Current position
SEEK_END	End of the file

Example: read the first character

```
FILE *fp = fopen("data.txt", "r");
if (fp != NULL)
{
    int ch;
    fseek(fp, 0, SEEK_SET);
    ch = fgetc(fp);
    if (ch != EOF)
        printf("%c\n", ch);
    fclose(fp);
}
```

rewind(fp) moves the file position back to the beginning. ftell(fp) reports the current file position as a long value (or -1L on error).

```
fseek(fp, 0, SEEK_END);  /* move to end */
long size = ftell(fp);    /* get position/size for a regular file */
rewind(fp);               /* return to beginning */
```

5. Reading and Writing Whole Lines

fgets() reads at most size - 1 characters and adds a terminating '\0'. It usually stops at a newline or at end of file.

```
char line[200];
while (fgets(line, sizeof(line), fp) != NULL)
{
    printf("%s", line);
}
```

fputs() writes a string to a file. It does not automatically add a newline.

```
fputs("Hello\n", fp);
fputs(line, outputFile);
```

Why use fgets()/fputs()?

• Use fgetc()/fputc() when a task is naturally character-by-character.

• Use fgets()/fputs() when processing lines, such as merging files or removing blank lines.

• Check the return value of fgets() against NULL to detect end of file or a read error.

6. Character-Testing and Conversion Functions

Include **<ctype.h>** to use the character functions below.

Function	Meaning	Example
isspace(ch)	Tests whitespace: space, tab, newline and others.	isspace((unsigned char)ch)
toupper(ch)	Converts a lowercase letter to uppercase.	ch = toupper((unsigned char)ch);
tolower(ch)	Converts an uppercase letter to lowercase.	ch = tolower((unsigned char)ch);
isalpha(ch)	Tests whether a character is a letter.	if (isalpha((unsigned char)ch))
isdigit(ch)	Tests whether a character is a digit.	if (isdigit((unsigned char)ch))

Important C detail: pass EOF separately and otherwise convert a possibly signed char to unsigned char before passing it to ctype functions.

7. Concept: Capitalize the First Letter of Each Word

Use this for Lab 22B-2 and 22C-1. The key is a flag variable, often called newWord.

```
int ch;
int newWord = 1;

while ((ch = fgetc(in)) != EOF)
{
    if (isspace((unsigned char)ch))
        newWord = 1;
    else if (newWord)
    {
        ch = toupper((unsigned char)ch);
        newWord = 0;
    }

    fputc(ch, out);
}
```

How the flag works

• At the beginning, newWord = 1 because the first character begins a word.

• When whitespace is found, set newWord = 1 again.

• At the next non-whitespace character, capitalize it and set newWord = 0.

• Keep all other characters unchanged.

Example

```
Input:  hello world
        welcome to c programming

Output: Hello World
        Welcome To C Programming
```

This capitalizes the first character only; it does not force the rest of the word to lowercase.

8. Concept: Count Characters, Spaces, Tabs and Newlines

```
int ch;
long chars = 0, spaces = 0, tabs = 0, newlines = 0;

while ((ch = fgetc(fp)) != EOF)
{
    chars++;
    if (ch == ' ')
        spaces++;
    else if (ch == '\t')
        tabs++;
    else if (ch == '\n')
        newlines++;
}
```

The total character count includes spaces, tabs, and newline characters. A text file's last line may not end with a newline, so newline count is not always the same as the number of visible lines.

9. Concept: Append and Copy Files

Append

• Open the destination file with mode "a".

• Open the source file with mode "r".

• Read each character from the source with fgetc().

• Write each character to the destination with fputc().

• Appending does not erase old destination content.

Copy

• Open the source with "r" and destination with "w".

• Copy character-by-character until EOF.

• Mode "w" overwrites an existing destination file.

• Always check both fopen() results before copying.

Common copy loop

```
int ch;
while ((ch = fgetc(src)) != EOF)
{
    fputc(ch, dest);
}
```

10. Concept: Reverse File Contents

A simple approach for a regular file is to seek to the end, use ftell() to get the end position, then seek to positions in reverse order.

```
fseek(fp, 0, SEEK_END);
long pos = ftell(fp);

while (pos > 0)
{
    pos--;
    fseek(fp, pos, SEEK_SET);
    int ch = fgetc(fp);
    if (ch != EOF)
        putchar(ch);
}
```

This reverses the byte/character order, including newline characters. As a result, a multi-line file may appear with its line endings reversed too. For a task that means reverse the order of lines but preserve each line, use a different line-based algorithm.

11. Concept: Merge Alternate Lines from Two Files

Use fgets() and fputs() because the unit being merged is a whole line.

```
char line[256];

if (fgets(line, sizeof(line), file1) != NULL)
    fputs(line, output);

if (fgets(line, sizeof(line), file2) != NULL)
    fputs(line, output);
```

Repeat these two operations in a loop until both files are at EOF. If one file has extra lines, write the remaining lines from that file.

Example

```
file1.txt:          file2.txt:
Apple               Red
Mango               Green
Orange              Blue

output.txt:
Apple
Red
Mango
Green
Orange
Blue
```

12. Concept: Delete Blank Lines and Insert a Blank Line

To determine whether a line is blank, assume it is blank and inspect each character. If any character is not whitespace, it is not blank.

```
int blank = 1;

for (int i = 0; line[i] != '\0'; i++)
{
    if (!isspace((unsigned char)line[i]))
    {
        blank = 0;
        break;
    }
}

if (blank)
    continue;  /* skip this line */

fputs(line, output);
```

To insert one blank line after the third nonblank line, increment a counter only after writing a nonblank line. When the counter reaches 3, make sure the third line ends with '\n', then write one extra '\n'.

13. Which functions belong to each Lab 22 task?

Lab question	Main functions / concepts
22A-1 Create/open/close	fopen(), fclose(), NULL check
22A-2 Count chars/spaces/tabs/newlines	fgetc(), EOF, counters, '\n', '\t'
22A-3 Demonstrate file functions	fprintf(), fscanf(), fputc(), fgetc(), fseek(), rewind()
22A-4 Append	fopen(..., "a"), fgetc(), fputc()
22A-5 Copy	fopen(..., "r"), fopen(..., "w"), fgetc(), fputc()
22B-1 Reverse contents	fseek(), ftell(), SEEK_END, SEEK_SET
22B-2 Capitalize words	isspace(), toupper(), flag variable
22B-3 Merge alternate lines	fgets(), fputs(), loop
22C-1 Capitalize words	Same concept as 22B-2
22C-2 Remove blank lines / insert line	fgets(), isspace(), fputs(), line counter

14. Common Mistakes and Fixes

• **Forgetting NULL check:** fopen() can fail if the file does not exist or permission is denied.

• **Using char for fgetc():** use int so EOF can be detected reliably.

• **Wrong mode:** "w" erases old content; use "a" to append.

• **Wrong filename or folder:** input.txt must be in the program's current working directory.

• **Forgetting fclose():** close each successfully opened file.

• **Writing to the input file by mistake:** use a separate output file when transforming content.

• **Missing newline:** fputs() does not automatically add one.

• **Not handling uneven files:** merging lines should continue until both files are finished.

• **Counting lines incorrectly:** count actual newline characters or nonblank lines according to the question's wording.

15. Practical Workflow: How to Run in VS Code / GCC

• Create a folder for Lab 22.

• Save one program at a time, for example lab22a1.c.

• Create its required input .txt files in that same working folder.

• Open the terminal in the correct folder.

• Compile with: gcc lab22a1.c -o lab22a1

• Run in PowerShell: .\lab22a1.exe

• Check output and inspect any created output file.

• Test with small input first, then test empty files and missing input files.

Example terminal commands

```
gcc lab22a1.c -o lab22a1
.\lab22a1.exe
```

16. Mini Practice Questions

• What is the difference between "w" and "a"?

• Why should fgetc() be stored in an int variable?

• What does EOF mean?

• What is the difference between fgetc() and fgets()?

• What do SEEK_SET and SEEK_END mean?

• Why do we use newWord = 1 in the capitalization program?

• Why does fputs() not guarantee a new line?

• How do you merge two files when one file has more lines than the other?

• What is the difference between a blank line and a line containing spaces?

• Which functions would you choose to count tabs and newline characters?

Reference note

This is a concept and reference guide prepared for the Lab 22 topics supplied by the student. It is not an official institution-issued manual. Follow your teacher's exact instructions if they define a task differently.